

StormBonk

Development & AI Design Essay

Memory / Essay — AI for Games

Team Name	Group 07 Games
Members	Miguel Requena López Marcos González Palazón Amine Saghrouchni Yousfi
Engine	Unity 6.3 (C#)
Genre	2D Top-Down Action Roguelike
Platform	PC (Windows)
Document ver	v1.0 — May 2026

0 Contents

1	Project Overview	2
2	Game Architecture	2
2.1	Core scripts	2
2.2	Overall AI architecture	2
3	AI System 1 — Fuzzy Logic (Unit 8)	3
3.1	Motivation	3
3.2	Design	3
3.3	Rules and defuzzification	4
4	AI System 2 — A* Pathfinding (Unit 3)	4
4.1	Motivation	4
4.2	Grid and node structure	4
4.3	Integration with enemies	5
4.4	Grid-based enemy spawning	5
5	AI System 3 — Hierarchical FSM (Unit 4)	5
5.1	Motivation	5
5.2	Two-level hierarchy	5
5.3	Blackboard integration in the HFSM	6
6	AI System 4 — Blackboard (Unit 7)	6
6.1	Design	6
6.2	Emergent coordination	6
7	Level and Wave Progression	7
7.1	Player progression	7
8	Challenges and Solutions	8
9	Conclusions	9

1 Project Overview

StormBonk is a 2D top-down wave survival game developed in Unity 6.3 as the practical project for the AI for Games subject. The player controls a mage who auto-fires at the nearest enemy and must survive successive waves of three distinct enemy archetypes: the **Melee Hunter**, the **Charger**, and a **Ranged Shooter** (whose projectile system is implemented in `MageProjectile.cs`). The game consists of 3 zones with 5 waves each, escalating in difficulty. The AI techniques documented in this essay apply to the Melee Hunter and Charger, the two archetypes with full AI scripts.

The central design goal was to demonstrate four AI techniques from the course curriculum: Fuzzy Logic (Unit 8), A* Pathfinding (Unit 3), Hierarchical Finite State Machine — HFSM (Unit 4), and the Blackboard coordination pattern (Unit 7). Each technique is applied to a concrete, observable game behaviour rather than as an isolated demo.

Beyond the core AI systems, the game was extended with three additional game-feel features: a **player progression system** (fireball damage, fire rate, and health improve after each completed wave), an **Arcane Pulse** (fully automatic defensive burst — `Physics2D.OverlapCircleAll` within 2.5u, triggers when ≥ 2 enemies are simultaneously in range, deals 20 damage to all enemies in the area, 4s cooldown), and a **grid-aware enemy spawner** that positions new enemies exclusively on walkable `PathfindingGrid` nodes — preventing out-of-bounds spawns and reusing the existing A* infrastructure for spawn-point selection. The initial fireball damage is 10 (base fire rate 0.65s) to reduce first-level difficulty; the auto-aim fireball was also fixed to spawn 0.5u offset toward the target and to continue in the last-known direction if the target is destroyed mid-flight. The `MageEnemy` state machine was revised to start directly in **Chase** state (eliminating the **Idle** gate), ensuring the mage always pursues the player regardless of spawn distance. The `PlayerHealthBar` was extended to keep `slider.maxValue` in sync with the player's `maxHealth` (which grows on level-up) and to force `fill.fillAmount = 1` at full health, correcting the visual gap caused by the Unity Slider handle offset.

2 Game Architecture

2.1 Core scripts

Script	Type	Responsibility
<code>Player.cs</code>	<code>MonoBehaviour</code>	Movement, health, auto-aim, fireball, Arcane Pulse
<code>MeleeHunter.cs</code>	<code>MonoBehaviour</code>	Fuzzy speed + A* pursuit + knockback
<code>Charger.cs</code>	<code>MonoBehaviour</code>	2-level HFSM + Blackboard + A* + knockback
<code>MageEnemy.cs</code>	<code>MonoBehaviour</code>	4-state FSM + A* pursuit/retreat + knockback
<code>EnemySpawner.cs</code>	<code>MonoBehaviour</code>	Timed enemy instantiation
<code>LevelManager.cs</code>	<code>MonoBehaviour</code>	Wave/level progression
<code>PathfindingGrid.cs</code>	<code>MonoBehaviour</code>	A* grid baking + path queries
<code>EnemyBlackboard.cs</code>	static class	Shared agent memory
<code>PlayerHealthBar.cs</code>	<code>MonoBehaviour</code>	HUD health visualisation

2.2 Overall AI architecture

Figure 1 shows how the four AI systems interconnect at runtime. The `PathfindingGrid` is queried by both enemy types for obstacle-avoiding paths. The `EnemyBlackboard` acts as the coordination layer: enemies write their intent (requesting a charge slot) and read global state (player position, stun flag). The `LevelManager` adjusts spawn rate per wave and receives kill notifications.

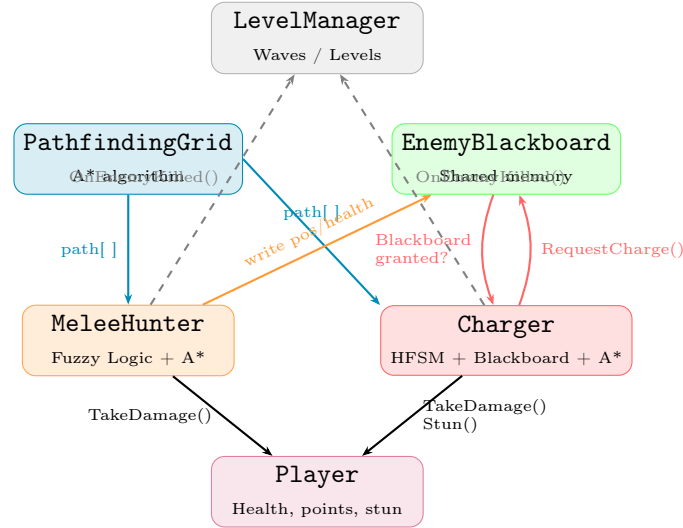


Figure 1: StormBonk AI system architecture at runtime.

3 AI System 1 — Fuzzy Logic (Unit 8)

3.1 Motivation

The original **MeleeHunter** moved at a constant `moveSpeed` regardless of context. This produced robotic behaviour: a nearly-dead enemy sprinted as fast as a fresh one. Fuzzy Logic (Unit 8) was applied to create smooth, context-aware speed modulation — an “area of grey” between the binary states of slow/fast.

3.2 Design

The system reads two inputs every frame:

- **Enemy health** — what percentage of its health the **MeleeHunter** has left (0 = dead, 1 = full health).
- **Player health** — what percentage of health the player has left.

Each input is mapped to two complementary fuzzy labels: *Healthy* and *Wounded* for the enemy; *PlayerHealthy* and *PlayerDying* for the player. The more health the enemy has, the more “Healthy” it is (and the less “Wounded”), and vice versa. Figure 2 shows this relationship as a graph.

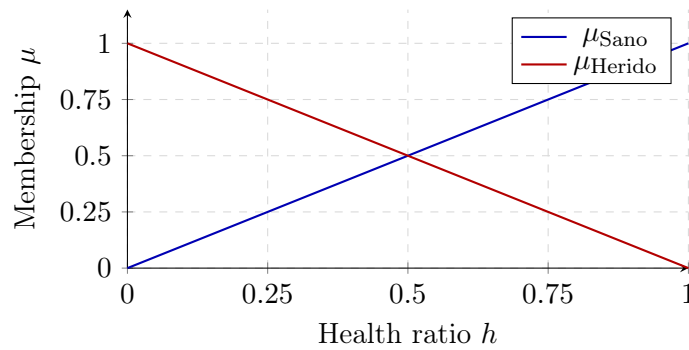


Figure 2: Linear ramp membership functions for hunter health. The two functions are complementary ($\mu_{\text{Sano}} + \mu_{\text{Herido}} = 1$).

3.3 Rules and defuzzification

Four plain rules combine both labels, each proposing a different movement speed:

Rule name	Condition	Proposed speed
Cautious	Enemy low health and player high health	0.8 u/s — backs away slowly
Normal	Enemy low health and player low health	2.0 u/s — advances carefully
Aggressive	Enemy high health and player high health	3.0 u/s — attacks decisively
Frantic	Enemy high health and player low health	5.0 u/s — maximum aggression

All four rules are evaluated at the same time and each receives a *weight* proportional to how strongly its condition is met. The final speed is a **weighted average** of the four proposed speeds: whichever rules are most satisfied pull the result toward their speed. If the player is stunned, a bonus multiplier is applied on top.

Listing 1: FuzzySpeed() — weighted centroid defuzzification

```

1 private float FuzzySpeed()
2 {
3     float h = (float)currentHealth / maxHealth;
4     float p = playerScript != null
5         ? (float)playerScript.currentHealth / playerScript.maxHealth : 1f;
6
7     float muSano = h; float muHerido = 1f - h;
8     float muSanoPJ = p; float muMuertoPJ = 1f - p;
9
10    float w1 = Mathf.Min(muHerido, muSanoPJ); // Cauteloso
11    float w2 = Mathf.Min(muHerido, muMuertoPJ); // Normal
12    float w3 = Mathf.Min(muSano, muSanoPJ); // Agresivo
13    float w4 = Mathf.Min(muSano, muMuertoPJ); // Frenetico
14
15    float total = w1 + w2 + w3 + w4;
16    if (total < 0.001f) return moveSpeed;
17
18    float speed = (w1*speedCauteloso + w2*speedNormal
19                  + w3*speedAgresivo + w4*speedFrenetico) / total;
20
21    if (EnemyBlackboard.PlayerIsStunned) speed *= stunnedBonus;
22    return speed;
23 }
```

4 AI System 2 — A* Pathfinding (Unit 3)

4.1 Motivation

Both `MeleeHunter` and `Charger` previously used direct `(target - self).normalized` movement vectors. In an arena with walls, this causes enemies to get stuck pushing against obstacles. Unit 3 establishes that A* (or NavMesh) is the standard solution.

4.2 Grid and node structure

`PathfindingGrid` divides the level into a grid of cells at scene start. Each cell stores:

- Whether it is **walkable** or an obstacle (detected with a 2D physics overlap).
- The **accumulated cost** from the starting cell.
- An **estimated cost** to the destination (heuristic).

- A **pointer** to the previous cell, used to reconstruct the path once the destination is reached.

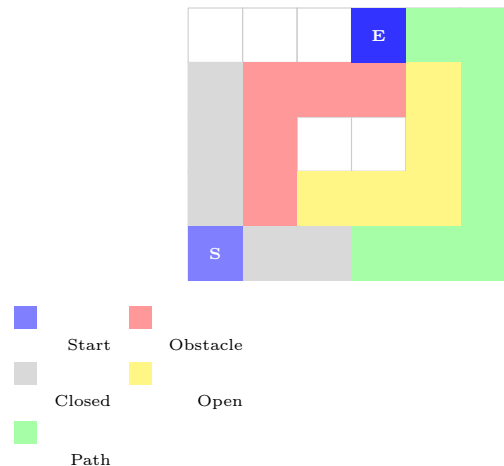


Figure 3: Example A* execution on a 6×5 grid. The path navigates around the wall obstacle (red cells).

4.3 Integration with enemies

All three enemies recalculate their path every fixed interval (0.4 s for the Hunter and MageEnemy, 0.3 s for the Charger) and follow the returned list of waypoints one cell at a time. **MageEnemy** uses A* for both *chasing* the player and *retreating*: in the retreat state it picks a destination point in the opposite direction of the player and finds a path there, automatically routing around walls. The **Charger** resets its path timer to zero when recovering from a stun, guaranteeing a fresh path calculation rather than following a stale route. If the grid is not present in the scene, enemies fall back to direct movement.

4.4 Grid-based enemy spawning

A secondary application of the same PathfindingGrid is **spawn-point selection**. The revised EnemySpawner calls:

```
PathfindingGrid.Instance.GetSpawnCandidates(center, minDist, maxDist)
```

which iterates all grid nodes and returns those that are (i) walkable and (ii) within the configurable distance ring $[minDist, maxDist]$ around the player. A random candidate is selected each spawn event. This guarantees enemies never materialise inside walls or beyond the arena boundary, while reusing the node data already baked at startup — no additional spatial structure is required.

5 AI System 3 — Hierarchical FSM (Unit 4)

5.1 Motivation

The original **Charger** used a flat **switch** over a single **State** enum (Idle / Prepare / Charge / Stunned / Dead). Unit 4 warns that flat FSMs become unmanageable as states grow. The Charger's death logic could fire mid-charge without properly releasing the Blackboard charge slot. An HFSM solves both problems elegantly.

5.2 Two-level hierarchy

The **Charger** has two levels of states:

- **Top level (TopState):** Alive or Dead.
- **Sub level (AliveState):** Approach → Prepare → Charge → Stunned.

The sub-level logic only runs while the top level is **Alive**. When the Charger dies, a single change of the top level to **Dead** immediately stops everything else, regardless of which sub-state was active.

The Charger starts directly in **Approach** as soon as it spawns, so it begins chasing the player from the very first frame.

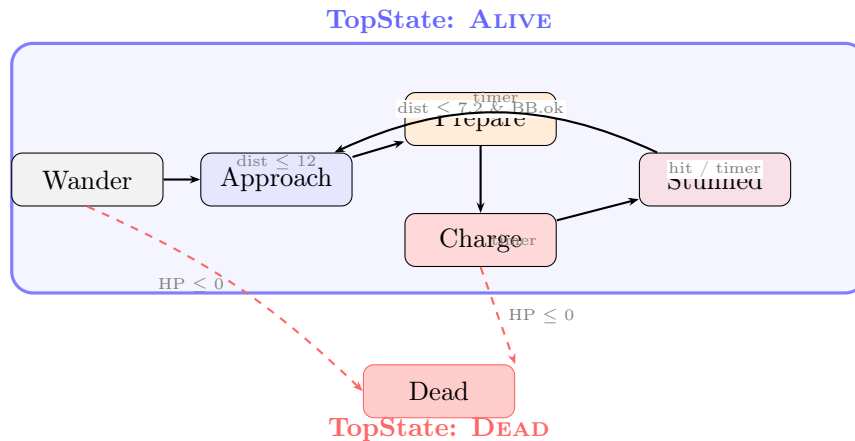


Figure 4: Charger HFSM. The outer box is **TopState.Alive**; dead is a top-level override that stops all sub-state processing.

5.3 Blackboard integration in the HFSM

The **Approach** sub-state adds a *Blackboard gate* before the transition to **Prepare**: only if `EnemyBlackboard.Request` returns `true` does the Charger begin its wind-up. This ensures that at most `MaxSimultaneousCharges` (default: 1) Chargers can charge at once. When the Charger enters **Stunned** or **Dead**, it releases the slot via `EnemyBlackboard.ReleaseCharge()`.

6 AI System 4 — Blackboard (Unit 7)

6.1 Design

The Blackboard pattern (Unit 7) provides a shared data board where agents can read and write information without communicating directly with each other. In StormBonk, `EnemyBlackboard` is a static C# class that acts as both the shared board and the arbitrator.

6.2 Emergent coordination

The most interesting behaviour arising from the Blackboard is the *stun cascade*:

1. A Charger hits the player and writes to the board that the player is stunned.
2. All `MeleeHunter` instances read that flag and increase their speed by 50% for as long as the stun lasts (1 s).
3. Result: a successful Charger attack triggers a coordinated rush by the Hunters, even though none of them “told” the others to do it.

This coordination was not explicitly scripted; it emerged from combining the Blackboard with Fuzzy Logic, showing the value of designing AI layers that complement each other.

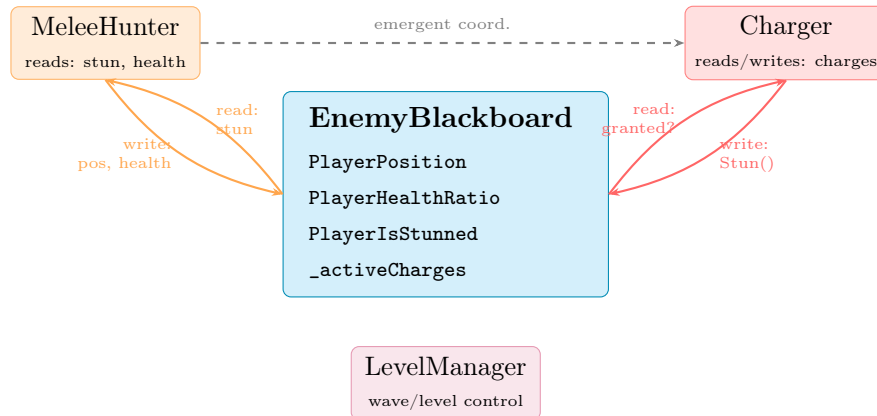


Figure 5: Blackboard pattern: **EnemyBlackboard** as central shared data store. Agents read/write without direct references to each other.

7 Level and Wave Progression

LevelManager implements the GDD’s 12-stage structure in a simplified 3-zone, 5-wave-per-zone form. On each `OnEnemyKilled()` call it checks whether the wave kill quota has been met, then either starts the next wave (with an increased spawn rate) or advances the level. The **EnemySpawner** receives the new `spawnRate` directly.

Wave	Kill quota	Spawn interval	Pressure
1	3	3.0 s	Introductory
2	6	2.5 s	Building
3	10	2.0 s	Medium
4	15	1.5 s	High
5	20	1.0 s	Maximum

The player HUD displays `WAVE X/5` and `LEVEL Y` via `TextMeshProUGUI` components updated by `LevelManager.RefreshUI()`. The player health bar (`PlayerHealthBar`) uses a colour gradient: green ($>50\%$ HP) $\rightarrow$ yellow $\rightarrow$ red ($<25\%$ HP), and also updates a numeric `hpText` label showing *HP current / max*.

7.1 Player progression

Each time the player completes a wave, `LevelManager.NextWave()` calls `Player.ApplyWaveProgression(wave, 1)` before starting the next wave. The method applies the following bonuses cumulatively:

Stat	Change	Cap / note
Fire rate	$\times 0.90$ per wave	min 0.25 s (base 0.65 s)
Fireball damage	+2 per wave	base 10; no upper cap
Move speed	+0.3 u/s on level start	max 6 u/s
Max HP	+10 on level start	starts at 100
Current HP	+20 per wave (partial regen)	capped at maxHP; full heal on level start

Additionally, the player emits an **Arcane Pulse** automatically whenever ≥ 2 enemies simultaneously enter a radius of 2.5 u: `Physics2D.OverlapCircleAll` hits all enemies in range, dealing 20 damage with a 4 s cooldown. Requiring a minimum of 2 enemies prevents the pulse from trivially clearing single targets. The Arcane Pulse replaces the former manual Space-bar melee strike

entirely — all player attacks are now fully automatic.

The progression model creates a two-sided difficulty curve: enemies become more numerous and spawn faster over time, while the player simultaneously grows more powerful — matching the expected power growth of a roguelite loop.

8 Challenges and Solutions

- 1. NavMesh in 2D Unity.** Unity's built-in NavMeshAgent is 3-D only. The NavMesh 2D workflow requires the optional `com.unity.ai.navigation` package and a NavMeshSurface baked on the XY plane — non-trivial for a student project. *Solution:* implemented a bespoke grid-based A* in `PathfindingGrid.cs` that works natively in 2D physics space.
- 2. Blackboard stale state.** `PlayerIsStunned` needed to auto-clear when the stun duration expired. *Solution:* `EnemyBlackboard.Tick()` compares `Time.time` against `_stunEnd` and clears the flag; called from `Charger.Update()`.
- 3. Fuzzy denominator zero.** When all four fuzzy weights are near zero (degenerate float state), a division-by-zero would occur. *Solution:* guard clause `if (total < 0.001f) return moveSpeed;`
- 4. Charge slot leak on instant death.** If a Charger died during the `Charge` or `Prepare` sub-state, its Blackboard slot was never released. *Solution:* `Die()` explicitly checks `_chargeGranted` and calls `ReleaseCharge()` before destroying the object.
- 5. Legacy health field removal.** `MeleeHunter` had a redundant `health` field alongside `currentHealth`. Removing it causes Unity to log a harmless serialisation warning on old prefabs. Documented in the setup guide.
- 6. Enemies spawning outside the walkable area.** The original `EnemySpawner` placed enemies at a random point on a circle around the player, which could intersect walls or fall outside the tilemap boundary. *Solution:* `PathfindingGrid.GetSpawnCandidates()` returns only walkable nodes within a configurable distance ring; the spawner picks randomly from this filtered list.
- 7. Charger remaining idle until the player approached.** The Charger's initial `Wander` state gated the transition to `Approach` on `dist ≤ visionRange`. Enemies spawned beyond that radius stood still in standby, which looked unnatural. *Solution:* the Charger initialises directly in `AliveState.Approach` and pursues the player unconditionally; the grid spawner already guarantees a valid starting distance.
- 8. Enemies getting stuck inside buildings.** `MageEnemy` used direct `MoveToward` for both `Chase` and `Retreat`, causing it to push indefinitely into walls when navigating around buildings. The Charger could also retain a stale A* path after recovering from a stun. *Solution:* `MageEnemy` now uses A* for all movement states (`Chase`, `Retreat`, `Attack` adjustment); the flee-point for `Retreat` is computed as a virtual target in the opposite direction of the player and fed to `FindPath`, so the path automatically routes around obstacles. The Charger resets `_pathTimer = 0` on every transition to `Approach`, forcing an immediate fresh path calculation on recovery.
- 9. MageEnemy remaining idle when spawned far from the player.** The original `MageEnemy` initialised in `State.Idle` and only transitioned to `Chase` when `dist ≤ detectionRange` (12 u). Enemies spawned at the arena's edges — or whose player moved away after spawn — stayed frozen indefinitely. *Solution:* the `Idle` state was removed entirely; the mage starts directly in `State.Chase` and never returns to `Idle`. The `detectionRange` field was also removed as it is no longer needed.

10. **Health bar showing a gap at full health.** Unity's `Slider` component offsets its fill to leave room for the handle widget, causing a visible gap at maximum value. Additionally, `slider.maxValue` was set once in `Start()` and never updated, so it desynchronised from `Player.maxHealth` when the player levelled up. *Solution:* `PlayerHealthBar.Update()` now resynchronises `slider.maxValue` with `_player.maxHealth` every frame, and directly forces `fill.fillAmount = 1f` whenever `currentHealth >= maxHealth`, overriding the slider's internal offset calculation.

9 Conclusions

The four AI techniques implemented in StormBonk address distinct layers of enemy intelligence:

- **Fuzzy Logic** eliminates the sharp speed discontinuities of a threshold-based system. The Melee Hunter's behaviour now feels organic — cautious when wounded, desperate when the player is near death.
- **A* Pathfinding** enables enemies to navigate around any static obstacle without per-level scripting. The fallback to direct movement preserves playability in scenes without a grid.
- **HFSM** reduced the Charger's death handling from a special-case scattered through multiple state handlers to a single top-level flag. Adding new combat sub-states in the future requires no changes to the death path.
- **Blackboard** produced emergent coordination — the stun cascade — without any hardcoded "team behaviour" logic. The same pattern can extend to future enemy types at zero coupling cost.
- **Grid-based spawning** extended the A* infrastructure beyond pathfinding: the same walkable-node graph used for enemy movement also determines valid spawn positions, preventing out-of-bounds materialisation with no additional spatial data structures.

Together, these five techniques demonstrate that well-chosen AI abstractions produce behaviour complexity that exceeds the sum of their individual parts, and that a single spatial data structure (the pathfinding grid) can serve multiple gameplay systems simultaneously.